

Defense Speech

Topic: Pricing American Options with a Neural Network Based on a Binomial Tree

Student: Artem Petrov

Supervisor: Sergey Shorokhov

Introduction

Good afternoon, dear members of the examination committee.

My name is Artem Petrov. Today I would like to present my final qualifying work. The topic of my work is: “Pricing American Options with a Neural Network Based on a Binomial Tree”.

The work is connected with mathematical finance, numerical methods, and neural networks. In this work I study American put options and implement a neural network architecture called BTNet.

Relevance

First, I would like to explain why this topic is important.

Options are widely used in financial markets. They are used for hedging risks, speculation, and portfolio management. A put option gives the right to sell an asset at a fixed strike price. An American option can be exercised not only at maturity, but at any time before maturity.

This feature makes American options more difficult to price than European options. For many European options we can use the Black-Scholes formula. But for an American put option there is no simple closed analytical formula in the general case. At every moment we must decide what is better: to exercise the option now or to continue holding it. Because of this, the pricing problem becomes an optimal stopping problem.

One classical method for this problem is the Cox-Ross-Rubinstein binomial tree model. In this model, the asset price can go up or down at each time step. The option price is calculated by moving backwards through the tree. For an American option, at every node we compare the continuation value with the immediate exercise value and choose the maximum.

The main idea of BTNet is to represent this binomial tree calculation as a neural network. The theoretical basis of this idea was proposed by Sergey Shorokhov. In his work, the backward induction of the binomial tree is represented as a forward pass of a special neural network.

I want to make clear that my work does not propose a completely new pricing theory. My contribution is practical and experimental. I implemented this architecture in Python and PyTorch, applied it to American put options, verified the results with QuantLib, and studied the behavior of the model in several experiments.

Goal and Tasks

The goal of my work is to implement and verify BTNet for pricing American put options.

To achieve this goal, I solved several tasks.

First, I studied the Black-Scholes model, the Cox-Ross-Rubinstein model, and the BTNet architecture.

Second, I implemented a software package called `bttn_bs`.

Third, I implemented two models: `BTNetEuropean` and `BTNetAmerican`.

Fourth, I compared the model prices with analytical Black-Scholes prices and with QuantLib.

Fifth, I studied the transfer of weights from the European model to the American model.

And finally, I checked how automatic differentiation can be used to compute option sensitivities.

Architecture and Implementation

Now I will describe the main idea of the architecture.

In the CRR model, option pricing is done by backward induction. In BTNet, the same calculation is written as a sequence of neural network layers. Terminal payoffs are represented by the first layer. Discounted risk-neutral averaging is represented by a linear filter. The sequence of backward induction steps is represented by several network layers.

For a European option, this structure is relatively simple, because there is no early exercise. For an American option, the model must also decide whether to continue or exercise. In `BTNetAmerican` this is done by maxout layers. The maxout operation chooses the maximum between the continuation value and the immediate exercise value. This is exactly the financial logic of an American option.

In the practical part, I implemented the package `bttn_bs`. It contains modules for layers, European and American models, analytical formulas, training, Greeks, and comparison with QuantLib. The main libraries are PyTorch, NumPy, SciPy, Matplotlib, and QuantLib.

The base experiment used the following parameters: initial asset price S_0 equals 100, maturity T equals 1, risk-free rate r equals 0.05, volatility σ equals 0.25, and tree depth n equals 9.

For verification, I used a grid of strike prices. For every strike price, I calculated the BTNet price and compared it with a reference value. For American options, I used QuantLib CRR with 500 steps as a numerical reference. It is important that this is not an exact analytical solution, but it is a good numerical benchmark.

Main Results

Now I will discuss the main results.

For the American put option with analytical CRR initialization, the model showed good accuracy. The mean absolute error was 2.84×10^{-4} . The RMSE was 4.06×10^{-4} . The maximum absolute error was 1.10×10^{-3} .

These errors are small for this experiment. They show that even a compact BTNet with tree depth n equals 9 can produce prices close to a much deeper QuantLib binomial tree with 500 steps.

The next experiment was connected with transfer of weights. The idea was simple. First, I trained the European BTNet on Black-Scholes prices. Then I transferred part of the learned weights to the American BTNet. The question was whether the European model can help the American model.

In the base scenario with σ equal to 0.25, the transferred weights did not improve the result. The mean absolute error increased from 2.84×10^{-4} to 4.38×10^{-4} . The RMSE and maximum error also increased.

After that, I tested several volatility scenarios: σ equals 0.10, 0.25, 0.60, and 0.90. The result was mixed. At low volatility, the two initializations were almost the same. At σ equal to 0.60, the transferred weights slightly improved the price error. But at high volatility, σ equal to 0.90, the transferred weights became much worse.

So the conclusion is that transfer of weights from the European model to the American model is not a stable method. It can help in one market regime, but it can also make the result worse in another regime. The analytical initialization from the CRR model is more reliable and easier to interpret, because it keeps the financial meaning of risk-neutral weights.

Sensitivities

I also checked option sensitivities, or Greeks. Greeks show how the option price changes when market parameters change. Delta is sensitivity to the asset price, Vega is sensitivity to volatility, Theta is sensitivity to time, and Gamma describes convexity.

Because BTNet is implemented in PyTorch, the model can use automatic differentiation. This is useful, because we can obtain sensitivities without deriving a separate formula for every case.

In my work, automatic differentiation gave useful results for Delta, Vega, and Theta. For American options, I compared the results with finite differences. At the same time, I found an important limitation of the current architecture. BTNet uses ReLU and maxout operations, and these operations are piecewise-linear. Because of this, Gamma becomes zero almost everywhere.

I do not focus on this as a failure of the model. It is better to view it as a known limitation and as a direction for further research. The pricing results are good, but for full risk management the architecture should be improved, for example by using smooth functions instead of ReLU and maxout.

Limitations and Future Work

There are also other limitations. The main experiments used fixed parameters S zero equals 0.5, T equals 1, r equals 0.05, and n equals 9. I tested several volatility values, but I did not cover all possible market conditions. I also did not separately study the influence of the tree depth n . In theory, a larger n should reduce the pricing error, but it also increases the size of the network and the computational cost.

The future development of this work may include several directions. First, smooth approximations such as Softplus, GELU, or soft maximum can be used to improve the behavior of sensitivities. Second, the model can be extended to local or implied volatility. Third, it may be applied to multidimensional assets or exotic options. Finally, the model can be calibrated on real market data.

Conclusion

To conclude, in this work I implemented and verified BTNet for American put option pricing. The model with analytical CRR initialization showed good pricing accuracy compared with QuantLib CRR with 500 steps. I also showed that transfer of weights

from a European model to an American model is not a reliable universal improvement.

The main practical result is a reproducible PyTorch implementation of an interpretable neural architecture for option pricing. The model combines the financial meaning of the binomial tree with the tools of neural networks and automatic differentiation.

Thank you for your attention. I am ready to answer your questions.

Approval

Approved by the English language tutor:

Full name: _____

Signature: _____

Date: “ ____ ” _____ 2026